FOR THE MAIN GITHUB ACCOUNT HOLDER

GitHub setup, step by step

So that Claude can propose changes, and nothing goes live until you say so.

At the end you will have: **Nobody** can change the live Radar by pushing code straight to it, not you and not Claude. Every change arrives as a **pull request**, a proposal you can read first. Three automatic checks run on every proposal: the API builds, the web app builds, and no passwords or keys are hiding in it. Claude writes proposals. **You** accept them.

You need: A GitHub login that **owns** the repository (you can see a Settings tab on it). The file **github-and-sandbox.zip**. The project folder **C:\dev\Rocket-crm** on your computer. About 60 minutes. Merging to the main branch goes live, because Railway and Vercel deploy from it.

Four words

Word	What it means
Repository	The project's home on GitHub.
Branch	A separate copy of the code to work on. The live one is called main.
Pull request	A proposal to join a branch into main, that people can read and approve.
Ruleset	The rules GitHub enforces on main.

The eight parts: 1 Check three things. 2 Put the files in. 3 Run the checks once. 4 Switch the rules on. 5 Test the rules. 6 Give Claude access. 7 Your first real change. 8 If something goes wrong.

PART 1

Check three things first

Five minutes. Do not skip this part: it decides whether the rest can work.

1. Is the repository **Private** or **Public**? Open your repository on github.com. Next to its name, near the top left, there is a small grey label that says Private or Public. Write down which.

2. Which plan do you have? Click your **profile picture** (top right), then **Settings**, then **Billing and plans** in the left list. Look at **Current plan**. Write it down.

> **Careful:** GitHub only enforces rules on a **Private** repository if you pay for a plan (GitHub Pro is the smallest). On the **Free** plan, rules work only on **Public** repositories. If yours is Private and the plan is Free, the rules in this guide cannot be switched on. Either upgrade (Billing and plans, then Upgrade), or stop here and tell Sushan. Do not carry on: Part 4 would fail.

3. Are you an admin? Look at the row of tabs on the repository page: Code, Issues, Pull requests, Actions, and so on. If you can see **Settings** at the end of the row, you are an admin. If not, this guide must be done by the main account holder.

4 Decide how **Claude** will appear on GitHub. Claude acts as whichever GitHub login you connect to it, and that decides who can approve its work. Read the table, then write down **A** or **B**.

	Path A (recommended)	Path B (simpler)
Claude's GitHub login	Its own, new, free login (for example rhino-claude)	Your own login
Pull requests show	The name rhino-claude	Your name
Who approves	You approve them, as yourself	Nobody can: GitHub does not let you approve your own pull request. Your click on Merge is the approval
How strong	Strongest. Claude cannot change or bypass any rule	Weaker. Only Claude's own limits stop it merging
Extra work	About 15 minutes, one extra login	None

Good to know: Why this matters: it is easy to assume Claude's pull requests come from a separate robot name. They do not. Claude Code on the web carries the GitHub login connected to it, so on Path B a pull request looks as if you wrote it.

PART 2

Put the files in the project

On your own computer, in PowerShell. Every command is on its own grey line: type or paste one at a time, and compare with "You should see".

1. Open PowerShell. Go to the project and make sure nothing is half finished.

```
cd C:\dev\Rocket-crm
git status
git branch --show-current
```

> **You should see:** working tree clean, then the word main. If it lists changed files, stop and ask Sushan.

2. Look at the folder names.

```
dir
```

> **You should see:** folders called API and Web, spelled with those capital letters. If the folder names are different, stop and tell Sushan.

3. Unpack the zip. It is in your Downloads folder.

```
cd $HOME\Downloads
Expand-Archive github-and-sandbox.zip pack -Force
```

> **You should see:** no message, and a new folder called pack in Downloads.

4. Copy everything into the project.

```
Copy-Item pack\* C:\dev\Rocket-crm -Recurse -Force
cd C:\dev\Rocket-crm
dir -Force
```

> **You should see:** folders named .github and docs, and a file called CLAUDE.md. This replaces three of your files with newer versions: API\src\index.ts, Web\src\components\Layout.tsx and Web\src\styles.css. That is expected.

5. Put your GitHub username in the code owners file. First find your username: it is the word after github.com/ in the address that this prints.

```
git remote -v
notepad .github\CODEOWNERS
```

You should see: In Notepad choose Edit, then Replace. Find what: YOUR-GITHUB-USERNAME. Replace with: your username. Click Replace All, save, close. Every line that starts with a slash now says @yourname.

6. Check exactly what changed.

```
git status
```

You should see: Modified: API/src/index.ts, Web/src/components/Layout.tsx, Web/src/styles.css. New: .github/, CLAUDE.md, docs/, API/src/lib/environmentGuard.ts, API/scripts/test-environment-guard.ts. If any OTHER file shows as modified, stop and send Sushan the list.

7. Check both apps still build.

```
cd API
npm run build
cd ..\Web
npm run build
cd ..
```

You should see: both finish without red error lines.

8. Save and send it. This is the **last time** you push straight to main.

```
git add -A
git commit -m "Add GitHub rules and sandbox safety"
git push
```

You should see: no error. Railway and Vercel start new deployments by themselves. Nothing visible changes in Radar. In the Railway log you should still see API listening and the Radar ordering connector line.

PART 3

Make the three checks run once

The rules wait for three checks. A check only exists after it has run once, so make them run before you switch the rules on.

1. Make a practice branch with one small file, and send it to GitHub.

```
git checkout -b practice-check
echo practice > docs\practice.txt
git add -A
git commit -m "Practice: make the checks run once"
git push -u origin practice-check
```

> **You should see:** a line ending in a web address for creating a pull request.

2. In your browser open the repository. A yellow bar says practice-check had recent pushes. Click **Compare & pull request**, then the green **Create pull request** button. (No yellow bar? Click the Pull requests tab, then New pull request, base main, compare practice-check.)

3. Scroll to the bottom of the page and wait 2 to 5 minutes.

> **You should see:** three lines: CI / API build, CI / Web build and CI / No secrets. Each gets a green tick.

> **Careful:** If a line goes red, click **Details** next to it, copy the red text and send it to Sushan. The usual causes: a missing package-lock.json file (create it by running npm install in that folder, then commit it), or a build script with a different name.

4. When all three are green, click **Close pull request** (not Merge), then **Delete branch**. Then tidy your computer.

```
git checkout main
git branch -D practice-check
```

> **You should see:** Switched to branch main.

PART 4

Switch the rules on

Now GitHub starts refusing changes that do not follow the rules.

1. Choose the rules file. **Path A**: protect-main.json. **Path B**: protect-main-solo.json. Both are in the folder .github\rulesets in your project.

2. In your browser: the repository, then **Settings** (top row), then **Rules** (left list), then **Rulesets**. Click the green **New ruleset** button, then **Import a ruleset**, and choose the file from step 1.

   > **You should see:** a form named Protect main (or Protect main (solo owner)) with Enforcement status set to Active.

3. Scroll to the bottom and click **Create**.

4. Check what was created. Click the ruleset to open it.

   > **You should see:** Target branches: Default branch. Rules: Restrict deletions, Block force pushes, Require linear history, Require a pull request before merging, Require status checks to pass with API build, Web build and No secrets. Path A shows 1 approval and code owner review. Path B shows 0 approvals.

> **Careful:** If GitHub says "Upgrade to GitHub Pro or make this repository public", you hit the plan problem from Part 1. Nothing is wrong with the file.

PART 5

Test that the rules really hold

Two tests, ten minutes. Shortcut: save check-rules.mjs as C:\dev\check-rules.mjs and run node C:\dev\check-rules.mjs inside the project. Finish with the same line plus the word cleanup.

Test 1: pushing straight to main must be refused.

1. Try to push a change straight to main.

```
cd C:\dev\Rocket-crm
echo test > rule-test.txt
git add rule-test.txt
git commit -m "rule test"
git push
```

> **You should see:** the push is REJECTED, with a message about "Repository rule violations". If it is NOT rejected, the rules are not on: go back to Part 4.

2. Undo it on your computer.

```
git reset --hard origin/main
```

> **You should see:** a line starting HEAD is now at.

Test 2: a bad change must be caught.

1. Send a change that contains an environment file.

```
git checkout -b rule-test2
echo TEST=1 > .env
git add -f .env
git commit -m "rule test 2"
git push -u origin rule-test2
```

2. On GitHub open a pull request for it (the yellow bar, as in Part 3).

> **You should see:** CI / No secrets goes RED (Details says the .env file is an environment file) and the Merge button is blocked.

3. Click **Close pull request**, then **Delete branch**. Tidy your computer.

```
git checkout main
git branch -D rule-test2
```

PART 6

Give Claude access

Follow Path A or Path B, whichever you chose in Part 1.

Path A: Claude gets its own GitHub login (recommended)

1. Create the login. Open a **private window** in your browser (Ctrl+Shift+N). Go to github.com and click Sign up. Use an email you can read, for example yourname+claude@gmail.com, and a username like rhino-claude. Verify the email. Keep this window for the next steps.

2. In your **normal** window (logged in as you): the repository, **Settings**, **Collaborators** (left list), **Add people**. Type the new username, pick it, click Add. (If the repository belongs to an organization, choose the role **Write**, never Admin.)

   > **You should see:** Pending invitation next to the name.

3. In the **private** window: open the invitation email, or go to github.com/notifications, and click **Accept invitation**.

   > **You should see:** the repository page opens for the new login.

4. In your **normal** window: install the Claude app on this repository. Go to github.com/apps/claude, click **Install** (or Configure), choose your account, choose **Only select repositories**, pick this repository, and click Install.

   > **You should see:** github.com/settings/installations lists Claude with your repository.

5. In the **private** window: go to claude.ai/code and sign in with YOUR Claude account. Click to connect GitHub. GitHub shows an Authorize screen for the new login: click **Authorize**. Then click **+ Select repo** and pick the repository.

   > **You should see:** the repository name appears and a session opens.

Careful: If the list says "No repos match": check the invitation was accepted and the app is installed on this repository. Then in claude.ai go to Settings, Connectors, disconnect GitHub and connect again. Still nothing? Use Path B.

6. Leave the setting **Allow unrestricted branch pushes** switched OFF in claude.ai/code. Then Claude can only push to branches whose names start with claude/.

Path B: Claude uses your own login

1. Install the Claude app yourself, exactly as in step 4 of Path A. Then go to claude.ai/code, connect your own GitHub, and pick the repository. Leave **Allow unrestricted branch pushes** OFF. Use the file **protect-main-solo.json** in Part 4.

   You should see: the repository appears in the list.

Good to know: On Path B the pull requests show your name, and your click on Merge is the approval.

PART 7

Your first real change

This proves the whole chain: Claude proposes, the checks run, you read, you merge.

1. Make a label once. On the repository: **Issues** tab, **Labels**, **New label**. Name it **sandbox** and click Create label. (It is used later, once the test copy exists.)
2. In claude.ai/code ask: **Add a file docs/hello.md with one friendly line. Follow CLAUDE.md. Open a pull request.**

   > **You should see:** a branch whose name starts with claude/, and a pull request.

3. On GitHub open the pull request. Read **What this changes**, **Why** and **Proof**. Click the **Files changed** tab and read the list of files.
4. Go back to the **Conversation** tab and wait for the three green ticks at the bottom.
5. **Path A only:** approve it. On the Files changed tab click the green **Review changes** button, choose **Approve**, click Submit review. (Path B: skip this step.)
6. Click **Squash and merge**, then **Confirm**, then **Delete branch**.

   > **You should see:** docs/hello.md is now on main. Railway and Vercel redeploy. Nothing visible changes in Radar.

> **Tip:** Every real change goes the same way: read it, wait for green ticks, approve (Path A), merge. Before you merge anything, look at the list of changed files. Be extra careful with any change to the folders .github and docs, and to CLAUDE.md: they hold the rules.

PART 8

If something goes wrong

Find what you see in the left column. When you ask Sushan for help, send: which Part and step you were on, a photo or screenshot of the screen or the red text, and whether you chose Path A or Path B.

What you see	What to do
The test push to main was NOT rejected	The rules are not active. Part 4: check Enforcement status says Active.
A check stays yellow and says Expected or Waiting	It never ran, or its name does not match. See the Actions tab. ci.yml must be on main (Part 2, step 8).
API build or Web build is red	Click Details. If it mentions npm ci or package-lock.json, run npm install in that folder and commit package-lock.json. Otherwise send Sushan the red text.
"Upgrade to GitHub Pro or make this repository public"	The plan problem in Part 1.
Claude's repository list says No repos match	Part 6, the box under step 5.
You cannot approve: "authors cannot approve their own pull request"	You are on Path B: the pull request is yours. Use the solo ruleset, or move to Path A.
You imported the wrong ruleset	Settings, Rules, Rulesets, click it, Delete ruleset, import the right file.
An emergency fix is blocked by the rules	Path A: use Merge without waiting for requirements on the pull request. Or set the ruleset to Disabled, fix it, and set it back to Active at once.